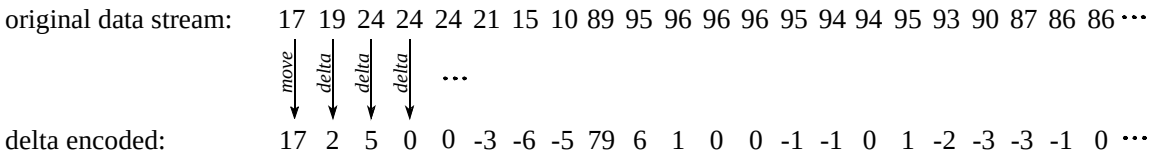

FIGURE 27-4
Example of delta encoding. The first value in the encoded file is the same as the first value in the original file. Thereafter, each sample in the encoded file is the difference between the current and last sample in the original file.

several techniques that store data as the *difference* between successive samples (or characters), rather than directly storing the samples themselves. Figure 27-4 shows an example of how this is done. The first value in the delta encoded file is the same as the first value in the original data. All the following values in the encoded file are equal to the difference (delta) between the corresponding value in the input file, and the *previous* value in the input file.

Delta encoding can be used for data compression when the values in the original data are *smooth*, that is, there is typically only a small change between adjacent values. This is not the case for ASCII text and executable code; however, it is very common when the file represents a *signal*. For instance, Fig. 27-5a shows a segment of an audio signal, digitized to 8 bits, with each sample between -127 and 127. Figure 27-5b shows the delta encoded version of this signal. The key feature is that the delta encoded signal has a *lower amplitude* than the original signal. In other words, delta encoding has increased the probability that each sample's value will be near zero, and decreased the probability that it will be far from zero. This uneven probability is just the thing that Huffman encoding needs to operate. If the original signal is not changing, or is changing in a straight line, delta encoding will result in runs of samples having the same value.

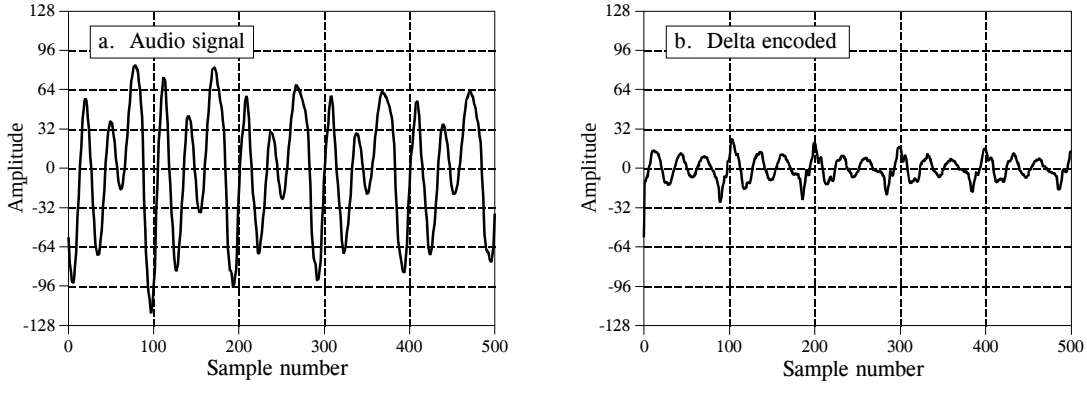

FIGURE 27-5
Example of delta encoding. Figure (a) is an audio signal digitized to 8 bits. Figure (b) shows the delta encoded version of this signal. Delta encoding is useful for data compression if the signal being encoded varies slowly from sample-to-sample.